Testing & Validation

Milestone 04 | Checking the current RenderX application

This report follows the project sequence: Milestone 01 set the requirements, Milestone 02 recorded the design and plan, Milestone 03 implemented the Learning Laboratory, and Milestone 04 checks the resulting application. It also explains how the current RenderX grew from the published Learning Laboratory.

1. What RenderX now does

RenderX began as a Python and Pygame Learning Laboratory for 3D wireframe graphics. The current version keeps the custom transform, camera, clipping and projection path, then adds a workspace, practical challenges, a three-stage maze and Breakpoint, a separate shooter. This milestone checks the current behavior and records what remains unverified.

Area	GitHub Learning Laboratory	Current RenderX
Main use	Explore and learn renderer steps through lessons, comparisons and a guided demo.	Explore meshes and complete three challenges, a wireframe maze campaign or Breakpoint.
Renderer	Custom Python wireframe transform and projection pipeline.	Retains the custom pipeline; adds hidden-line, solid and depth-oriented software modes.
Models and files	Procedural teaching shapes and sample models.	Adds bounded OBJ import, recent files, readable errors and showcase models.
Validation	Published README reports 86 tests; this is a claim in the old README, not a rerun here.	Current automated suite: 135 tests passed. Focused edge, resize, scenario and performance checks are also recorded below.

Table 1. Summary of the change from the published Learning Laboratory to the current application.

The comparison uses GitHub main at 972266d (25 Sep 2026) and the current application snapshot at d34e273 (23 Oct 2026). Their Git histories are unrelated, so this is a source and feature comparison, not a single commit chain.

The tree comparison counted 423 changed paths, including source, archived files, assets, screenshots and evidence. That is not a count of 423 user features.

2. System architecture

The Explore, Challenges and Maze screens share the Python renderer. For a frame, the app reads input, updates the scene, transforms mesh points, clips edges near the camera, projects visible points and draws them with Pygame. Breakpoint runs in a separate OpenGL 3.3 window.

Input and pages	Scene and model	3D pipeline	Display
Pygame events; Explore, Challenges, Maze	Vertices, edges, transforms, selected OBJ	Transform, camera, clip, project	Software wireframe and raster styles

Figure 1. RenderX software rendering path

Breakpoint uses its own OpenGL models, shaders and HUD. Its real graphics window cannot be created by the SDL dummy display used for the automated app screenshots.

Part	Responsibility	Example source
Main UI	Landing, navigation, panels, preferences and file actions.	renderx/dashboard.py; renderx/ui/
Geometry and renderer	Mesh data, transforms, camera, clipping, projection and drawing.	renderx/core/; renderx/engine/
OBJ loader	Reads a bounded subset of vertices and polygon faces.	renderx/core/obj_loader.py
Maze and challenges	Campaign state and renderer-based exercises.	renderx/maze.py; renderx/quest.py
Breakpoint	Separate OpenGL shooter and game state.	renderx/shooter/

Table 2. Main application parts included in the validation scope.

3. Technology stack and test environment

The test run used the project virtual environment and SDL's dummy video driver for automated window checks. This lets the app loop run without a visible desktop window; it does not reproduce a GPU display or native Windows dialogs.

Item	Observed or required value
Operating system	Windows 11, build 10.0.26300
Python	3.13.15 (project supports Python 3.10 and later)
Pygame	2.6.1
Test runner	Python unittest discovery; 135 test methods in the full run
Window sizes	800x500, 1024x640, 1280x720, 1366x768, 1920x1080, 2560x1440, 1024x1600 and 2200x720
Shooter requirement	A graphics device with OpenGL 3.3 support for interactive Breakpoint gameplay
Not recorded	The fresh automated run did not record the GPU model or a native desktop frame rate.

Table 3. Software and conditions for the recorded validation run.

The system version and Python/Pygame versions were read from the machine used for this report. The window-size values are the sizes requested by the QA runner, not display resolutions supported by every monitor.

The application keeps preferences and best scores in local files. It does not need a database or an account server.

4. Test plan and execution

A unit test checks one small function or class. An integration test checks connected parts. A system check drives several parts through a user-like flow. A regression test repeats a check after a fix to make sure the old problem stays fixed.

Level	What it checks in RenderX	How it was run
Unit-focused	Projection, clipping, mesh and OBJ rules, font layout, icon bounds and settings.	135-test suite, grouped by the test modules' main purpose.
Integration-focused	Navigation, file open, saved settings, game state and UI actions.	Same full test suite; module grouping is described in Appendix A.
System / headless	Launch, destinations, file feedback, shortcuts and window resizing.	Real app loop with scripted input and screenshots.
Boundary checks	Zero/negative depth, non-finite values, broken OBJ and oversized OBJ.	12 focused engine and input checks.
Performance	Average frame time and process working set for three Explore scenes.	Three paced 5-second dummy-display runs.

Table 4. Test levels used for this milestone.

The full suite ran with SDL_VIDEODRIVER=dummy and the app settings redirected to a temporary folder. The run finished with 135 tests and no failures. Separate QA results are reported in Sections 6 and 8.

The unit/integration split is a report grouping by test module: six focused modules account for 49 unit-focused tests; the remaining application-flow modules account for 86 integration-focused tests. The full suite was run together.

5. Unit and integration testing

The test suite checks the renderer's math as well as the code that joins the renderer to the desktop interface. The table shows representative checks, not all 135 test methods.

ID	Area	Input or action	Expected and recorded result
UT-01	Mesh data	Build built-in shapes and inspect edge indices.	Valid meshes and expected counts; passed.
UT-02	Transform math	Rotate known points around X, Y and Z.	Known results and unchanged source geometry; passed.
UT-03	Projection	Compare perspective and orthographic points at two depths.	Perspective size changes; orthographic size does not; passed.
UT-04	Clipping	Place an edge across the near plane.	Visible segment remains; behind-camera points are rejected; passed.
UT-05	OBJ parser	Read triangle, quad, face variants and comments.	Boundary edges are unique and vertices normalize; passed.
UT-06	Invalid values	Try empty, non-finite and out-of-range OBJ data.	Controlled ValueError; no partial scene change; passed.
UT-07	Icon layout	Draw SVG and mesh icons from 12 to 32 pixels.	Pixels stay inside the target and remain centered; passed.
UT-08	Small text	Use minimum supported controls and shortcut overlay.	No clipped binding or unsupported ellipsis; passed.

Table 5. Representative unit-focused checks.

Listing 1. Quad face becomes four unique edges

tests/test_extensions.py, lines 78-85

```
    def test_polygon_closure_unique_edges_and_normalization(self):
        text = "v 10 20 0\nv 14 20 0\nv 14 24 0\nv 10 24 0\n"
        mesh = self.load_text(text + "f 1 2 3 4\nf 4 3 2 1\n")
        self.assertEqual(mesh.edges, [(0, 1), (0, 3), (1, 2), (2, 3)])
        for actual, expected in [(mesh.vertices[0], (-1, -1, 0)),
                                 (mesh.vertices[2], (1, 1, 0))]:
            for value, target in zip(actual, expected):
                self.assertAlmostEqual(value, target)
```

The test also checks that the imported vertices are normalized.

5.1 Integration checks

Integration testing checks that an action reaches the right part of the app and leaves the rest of the state consistent.

ID	Action	Expected result	Recorded result
IT-01	Open bundled house.obj.	Scene selection, model list and recent file update together.	Passed: tests/test_file_open.py.
IT-02	Open a missing or invalid OBJ.	Readable error; old model and selection remain.	Passed: tests/test_file_open.py and edge runner.
IT-03	Click destination tabs and cycle with Tab.	Explore, Challenges, Maze and Shooter state follows the active page.	Passed: dashboard and launch-flow tests.
IT-04	Change volume and mute music.	Preference state persists without overlapping compact controls.	Passed: audio and dashboard tests; physical playback not covered.
IT-05	Collect keys and reach a Maze exit.	Locked exit opens when objectives are complete.	Passed: maze movement/state tests.
IT-06	Exercise Shooter state and launch failure.	Rules stay bounded; failed GL launch returns to RenderX.	Passed in tests; live GPU combat not covered.

Table 6. Representative integration checks.

Listing 2. Opening an OBJ updates the scene and recent list

tests/test_file_open.py, lines 20-31

```
    def test_open_obj_adds_selects_and_remembers_mesh(self):
        scene, models = Scene(), make_library()
        with tempfile.TemporaryDirectory() as temporary:
            settings = Path(temporary) / "settings.json"
            with patch("renderx.theme._SETTING", settings), \
                    patch("renderx.theme._recent_files", []):
                result = load_obj_file(SAMPLE_OBJ, scene, models)
                self.assertEqual(result, "Opened house.obj")
                self.assertEqual(scene.shape, "house")
                self.assertIn("house", models)
                self.assertEqual(json.loads(settings.read_text())["recent_files"],
                                 [str(SAMPLE_OBJ.resolve())])
```

6. System testing and regression checks

System testing follows the app from launch through its main destinations. The screenshot harness launched the current app, injected scripted events where needed, checked the final state and saved the Pygame display surface.

Step	User-like action	Evidence
1	Launch RenderX.	Landing screen opens; launch-flow tests check the default route.
2	Enter Explore and inspect the built-in model.	Cube, inspector, scene list and status capture.
3	Open a sample OBJ and an oversized OBJ.	House appears; oversize input shows controlled feedback.
4	Open menu, shortcuts, preferences and context actions.	Captured screens and input-flow tests pass.
5	Visit Challenges and Maze; inspect Shooter destination.	All three page shells are captured; Breakpoint gameplay is not.
6	Resize the app and close/return through launch routes.	Eight sizes and 50 rapid resize events passed.

Table 7. System-level flow exercised by the app harness.

800 x 500	1024 x 640	1280 x 720	1366 x 768
Pass	Pass	Pass	Pass
1920 x 1080	2560 x 1440	1024 x 1600	2200 x 720
Pass	Pass	Pass	Pass

Table 8. Eight requested window sizes; the run also injected 50 rapid resizes.

Regression checks repeated navigation, focus-loss handling, file-open errors, icon bounds, tab width, current model status and launch defaults after the relevant fixes. FEATURES.md was used as the current feature checklist.

7. Defects found and fixed

BUGLOG.md contains 11 recorded findings. Severity labels below are report-level impact estimates: high means a possible crash or invalid projection; medium means a blocked interaction; low means a visual or test-reliability issue.

ID / impact	Reproduction and observed result	Root cause	Fix, check and status
RX-UI-001 Low / UI	Open Explore's object list. Each row showed the same generic box instead of a distinct mesh preview.	ScenePanel reused one fixed mesh icon, and the top bar used the generic box as the brand.	Added the RX mark and mesh previews. Headless screens show them. Fixed (checked).
RX-UI-002 Low / window	Inspect the app window identity. The default runtime icon remained visible.	The app set window captions but did not set a project icon.	Added a shared identity helper. RenderX was checked; Breakpoint's OpenGL window is unavailable in dummy mode. Fixed (RenderX checked; Breakpoint icon unverified).
RX-CORE-001 High / projection	Project a point at depth zero with a zero near plane. It raised ZeroDivisionError instead of rejecting the point.	The guard compared depth only to the supplied near plane, so zero passed when that plane was zero.	Reject non-positive depth before division. Edge checks passed for zero, negative and clipped positive depth. Fixed (checked).
RX-CORE-002 High / projection	Project extreme but finite coordinates. The result contained an infinite screen coordinate.	Input values were checked, but projected output was not checked after scaling.	Reject non-finite output coordinates. Engine-edge checks passed. Fixed (checked).
RX-UI-003 Medium / input	Open keyboard help and click a destination beneath it. The destination changed while help stayed open.	The overlay did not capture mouse input or consume the outside click.	Help now captures and consumes the click. Replayed event sequences passed. Fixed (checked).
RX-UI-004 Low / focus	Hover or press a control, then leave the window or lose focus. Its transient style stayed active.	Leave and focus-loss handlers did not clear hover and pressed state.	Clear transient state on leave and focus loss. Replayed event sequences passed. Fixed (checked).

Table 9. Recorded defect register, part 1.

The projection fixes protect two different points in the pipeline. One stops unsafe depth before division; the other catches overflow after projection. The engine edge runner rechecked both.

7.1 Defect register, continued

ID / impact	Reproduction and observed result	Root cause	Fix, check and status
RX-UI-005 Low / pointer	Move over a button or field. The pointer remained an arrow instead of showing the control type.	Dashboard hit testing had no cursor mapping.	Added cursor requests. Event checks saw hand then arrow; native cursor appearance was not available in dummy mode. Fixed (requests checked; desktop appearance unverified).
RX-UI-006 Low / layout	At 1024x640, inspect a narrow icon-led control. The ellipsis could overlap its icon.	Text truncation always drew an ellipsis even when it could not fit.	Omit the label when the mark cannot fit. Typography audit passed at the compact size. Fixed (checked).
RX-UI-007 Low / pointer	Press and hold a button, then leave the window before releasing it. The pressed state remained.	The leave handler cleared hover but not the pressed action.	Clear the pressed state on leave. Replayed event schedule passed. Fixed (checked).
RX-UI-008 Medium / text	Draw a long label that truncates. The bundled font raised an error on the ellipsis glyph.	The font did not contain the Unicode ellipsis character.	Use three ASCII periods. Focused font checks and the full suite passed. Fixed (checked).
RX-TEST-001 Low / tests	Open files before running the recent-file test. The test inherited old paths and could fail its one-item check.	The test redirected the settings file but did not reset the in-memory recent list.	Reset the in-memory list inside the test. The fresh full suite passed. Fixed (checked).

Table 10. Recorded defect register, part 2.

A useful example came from the test-isolation defect. OBJ opening worked, but the test could fail after the app had already stored recent files. The fix resets that in-memory list for the test, then the whole suite was run again with settings redirected to a temporary folder.

No issue is marked open in BUGLOG.md. Two native-only details remain outside this check: the cursor's visible shape on a real desktop and Breakpoint's own OpenGL window icon.

What we learned

Cause group	Example from this review	Lesson
Hard-coded sizes	Icon spacing and narrow Challenge labels could conflict with controls.	Measure text and available control space before drawing.
Old data not refreshed	Torus was visible while the status bar showed Cube counts.	Build status from the currently selected model.
Missing input checks	Zero depth, extreme projection values and malformed OBJ data.	Validate values before division, drawing or changing the scene.

Table 11. Lessons grouped by the cause patterns found during review.

7.2 Checks raised by earlier interface feedback

These checks are included because they were raised during project review. They are separate from the 11 BUGLOG.md entries; the report does not add an undocumented root cause.

Earlier observation	Current evidence	Status
Icon looked clipped or off-center.	Current pixel-bound test checks every SVG and mesh icon at five sizes; current toolbar capture is included.	Current layout passes. No matching before image was available.
Challenge tab shortened to 'Chall...'.	Launch-flow test checks actual tab labels and page shortcuts.	Current tab text fits. No matching before image was available.
Status bar showed Cube counts while Torus was visible.	Historical screen paired with current Torus capture; a test compares status to the active shape.	Current status check passes.
App opened in Shooter by default.	Landing screen capture and launch-flow tests check the initial destination.	Current app opens on Landing; Enter opens Explore.
Left and right panels seemed to repeat headings.	Current Explore capture shows SCENE and INSPECTOR as separate panel headings.	Not reproduced in the current screen; no earlier panel capture found.

Table 12. Earlier UI observations checked against current evidence.

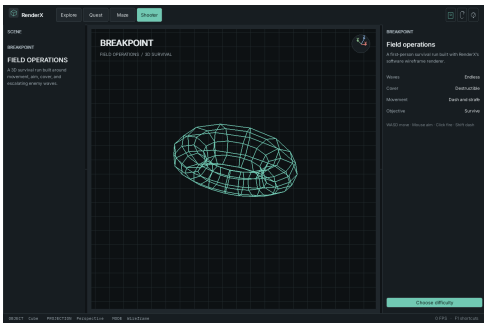


Figure 2. Earlier capture: Torus shown with stale Cube status.

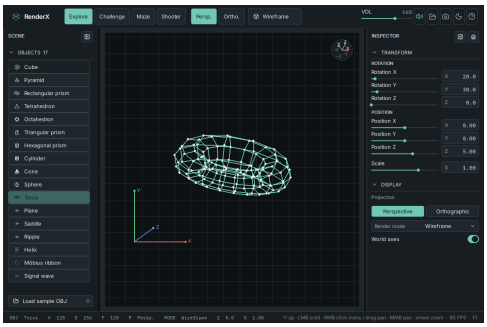


Figure 3. Current capture: Torus mesh with matching status.

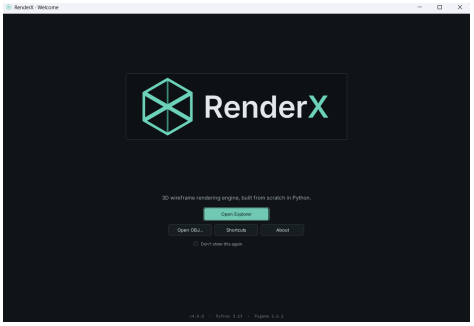


Figure 4. Earlier landing screen with the misaligned option.

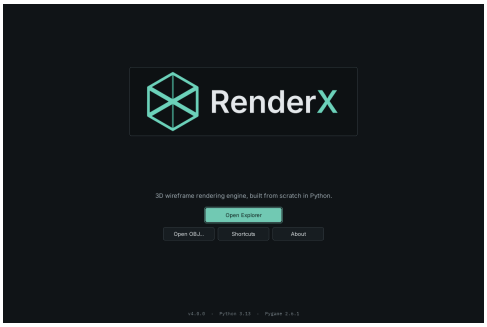


Figure 5. Current landing screen after the option was removed.

8. Validation and results

F01-F10 are the feature IDs from the supplied Milestone 01 PRD. The table checks those original aims against current tests. The later v4 features are listed separately because they were added after that requirements sheet.

ID	Milestone 01 acceptance area	Current check	Status
F01	Launch, use and close the app.	Landing and exit routes; launch-flow tests.	Pass in tested routes
F02	Valid built-in mesh data.	Shape and mesh validation tests.	Pass
F03	Perspective projection and safe depth.	Projection, clipping and engine-edge checks.	Pass for listed cases
F04	Three-axis rotation from input.	Known rotations and interaction tests.	Pass
F05	Move a model without mutating source points.	Translation and original-vertex tests.	Pass
F06	Uniform scale with positive bounds.	Scale interaction and bounds tests.	Pass
F07	Choose either built-in shape.	Dashboard selection and model tests.	Pass
F08	Reset selected model transforms.	Scene reset and UI action tests.	Pass
F09	Readable controls and current status.	Dashboard tests and visible status capture.	Pass in tested paths
F10	Input remains usable after focus changes.	Focus, leave-window and release-event tests.	Pass

Table 13. Validation against the original feature IDs.

The acceptance table shows the requirements that still matter in the current app. It does not claim a full manual acceptance run on every feature of the later shooter.

8.1 Later features added after the original requirements

Current feature	Evidence	Status
OBJ import and safe errors	Parser and file-open tests; over-limit screenshot.	Pass for bounded supported subset; native picker not checked.
Challenges and Maze	Quest and Maze tests; destination captures.	Pass in tested code paths.
Breakpoint shooter	Game-rule tests and launch-failure recovery.	Partly checked; live OpenGL run remains unverified.
Audio settings	Preference and audio state tests.	State passes; physical device playback not checked.
Responsive desktop layout	Eight sizes and 50 rapid resize events.	Pass for the tested size matrix.

Table 14. Validation of later additions without assigning them old requirement IDs.

Performance testing

The performance runner used three paced 5-second SDL dummy-display runs, with 300 target frames per run. The FPS column is 1,000 divided by average frame time; it is a headless comparison value, not a graphics-card benchmark.

Scene	Mean frame time	p95	p99	Reciprocal FPS	Peak working set
Explore idle	6.565 ms	8.211 ms	8.903 ms	152.3	65.11 MB
Explore orbit	6.511 ms	7.703 ms	8.618 ms	153.6	65.12 MB
Signal wave mesh	10.642 ms	11.930 ms	12.605 ms	94.0	65.37 MB

Table 15. Fresh 5-second headless measurements, 23 October 2026.

Peak resident memory rose by about 9.8-10.0 MB over the run baseline. The tests did not record minimum instantaneous FPS, native GPU FPS, or the parse time for a successful large external OBJ.

8.2 Input safety and basic security

RenderX is an offline desktop app. It has no account service, server API or network-based user data. The relevant input boundary in this milestone is a local OBJ file, plus saved preference and score files.

Check	Current protection	Evidence / limit
File size	OBJ reader reads at most 2,000,001 bytes before rejecting over 2,000,000 bytes.	Oversized fixture and visible error scenario passed.
Geometry size	Limits of 20,000 vertices and 50,000 unique edges.	Loader and edge tests cover limits.
Numeric values	Reject non-finite vertex values and unsafe projected results.	Core and engine-edge checks passed.
Face references	Reject empty, malformed and out-of-range face data.	OBJ parser and failure-preservation tests passed.
File execution	OBJ records are parsed as text; they are not evaluated as Python code.	Source review of the current loader; no penetration audit.
Network threats	No login or network endpoint exists in this desktop app.	Not applicable to the current architecture.

Table 16. Basic input checks carried out for the current offline app.

A corrupt or oversized file should produce an error while leaving the current scene available. This is checked by the parser tests, the engine-edge runner and the oversized-input screenshot scenario.

This is basic local-file robustness testing, not a formal security review. Authentication, network attacks, deployment, malware scanning and third-party penetration testing were not performed.

8.3 Summary, limits and next work

The fresh full suite completed with 135 tests passing in 11.507 seconds. No test failed. The edge runner passed 12 checks; the resize matrix passed 8 sizes and 50 rapid events. The screenshot harness captured 15 current app scenarios. The performance runner completed all three scenarios.

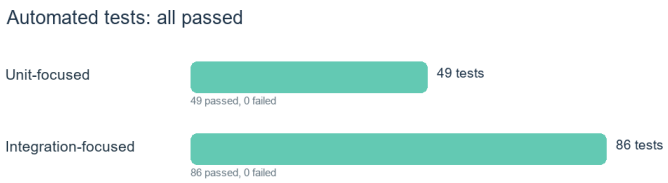


Figure 6. Automated tests grouped by module purpose; the combined run passed all 135.

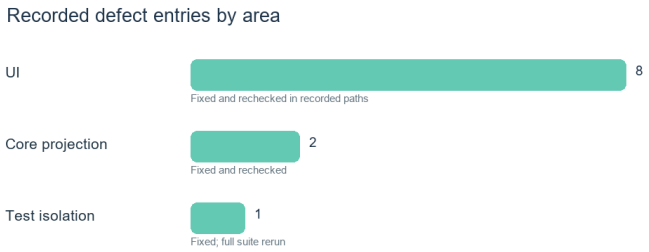


Figure 7. Eleven documented defects: eight UI, two projection and one test-isolation.

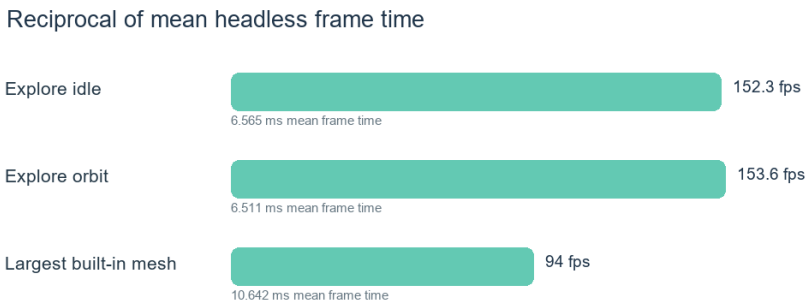


Figure 8. Reciprocal of mean frame time in the headless runs; this is not a native GPU benchmark.

Not measured or not confirmed

A live Windows acceptance session, Breakpoint gameplay on a real OpenGL display, the native file chooser, physical audio output, visible system cursor, fullscreen across monitors, minimum instantaneous FPS and a successful parse of the student's large model.obj were not confirmed. The user previously reported that model exceeded 2 MB, but the file was not present during this run. The loader's documented limit is 2 MB.

Next work

Before the demonstration, run the app once on the target laptop: open a normal OBJ, try the native file dialog, play a short Breakpoint run, confirm music and effects, resize the window and close it. Simplify or split any external OBJ above the documented limit.

References and test file map

Project sources

- RenderX public GitHub repository, baseline commit 972266d8ff9582fbb9f2f81fd65e855cfb635b0a, 25 September 2026.

- Current RenderX code snapshot d34e273b5d9699c915589e5429f0bf27e84401da, 23 October 2026.

- Project README.md, FEATURES.md, BUGLOG.md and docs/TESTING.md.

- Milestone 01 PRD, feature IDs F01-F10, supplied as read-only project source.

Testing tool

- Python unittest: standard library test framework used for the fresh run.

Test modules

Main purpose	Test files
Geometry and renderer	test_core.py, test_engine.py, test_comparison.py, test_extensions.py
UI, input and launch	test_dashboard.py, test_input_map.py, test_interaction.py, test_launch_flow.py, test_ui_fonts.py, test_window.py
File and settings	test_file_open.py, test_settings_path.py
Challenges and games	test_quest.py, test_maze.py, test_shooter.py, test_audio.py

Table 17. Current test modules included by unittest discovery.

No database was introduced. The application uses local preference and score files, as documented in the current project.

Appendix A | Re-running the checks

Run these commands from the repository root in PowerShell. The temporary settings path keeps the test run separate from normal RenderX preferences.

Listing 3. Commands used for the current report

PowerShell, from the repository root

```
$env:LOCALAPPDATA = Join-Path $env:TEMP 'renderx-m04-fresh-validation'
$env:SDL_VIDEODRIVER = 'dummy'
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe tools\qa\engine_edges.py
.\.venv\Scripts\python.exe tools\qa\resize_matrix.py
.\.venv\Scripts\python.exe tools\qa\performance_runs.py --seconds 5 --output
.\.venv\Scripts\python.exe output\milestone_04\source\capture_screenshots.py
```

Recorded fresh outputs

Run	Result
unittest discovery	Ran 135 tests in 11.507 seconds; OK.
engine_edges.py	12 named geometry and OBJ checks passed.
resize_matrix.py	8 size snapshots passed; 50 rapid resize events; no runner error.
performance_runs.py	Idle, orbit and largest-mesh scenarios passed; 300 target frames each.
capture_screenshots.py	15 current application scenarios captured with passed runner status.

Table 18. Fresh recorded checks for this version of the report.

A screenshot of the Shooter destination is only a headless app-shell capture. It is not evidence that live Breakpoint combat ran on a GPU.

Appendix B | Code listings

Listing 4. Icon pixels stay inside the button rectangle

tests/test_launch_flow.py, lines 176-194

```
    def test_every_svg_and_mesh_icon_is_safe_at_small_and_large_sizes(self):
        icon_names = [path.stem for path in icons._ASSET_DIRECTORY.glob("*.svg")]
        icon_names.extend(f"shape:{name}" for name in make_library())
        icon_names.append("brand")
        for name in icon_names:
            for side in (12, 16, 20, 24, 32):
                target = pygame.Surface((side + 32, side + 32), pygame.SRCALPHA)
                rect = pygame.Rect(16, 16, side, side)
                icons.draw_icon(target, name, rect, (245, 245, 245))
                pixels = target.get_bounding_rect(min_alpha=1)
                self.assertFalse(pixels.colliderect(pygame.Rect(0, 0, 16, 16)))
                self.assertGreaterEqual(pixels.left - rect.left, 1, (name, side))
                self.assertGreaterEqual(pixels.top - rect.top, 1, (name, side))
                self.assertGreaterEqual(rect.right - pixels.right, 1, (name, side))
                self.assertGreaterEqual(rect.bottom - pixels.bottom, 1, (name, side))
                self.assertLessEqual(abs(pixels.centerx - rect.centerx), 1,
                                     (name, side, pixels, rect))
                self.assertLessEqual(abs(pixels.centery - rect.centery), 1,
                                     (name, side, pixels, rect))
```

This checks five icon sizes and the four pixel margins around the rendered mark.

Listing 5. OBJ size and numeric input guards

renderx/core/obj_loader.py, lines 36-52

```
def load_obj(path: str | Path) -> Mesh:
    """Parse a safe subset of OBJ files.

    Extracts vertices 'v' and polygonal faces 'f', converts faces into boundary edges,
    and normalizes the mesh geometry into a unit bounding cube.
    """
    with Path(path).open("r", encoding="utf-8-sig") as source:
        text = source.read(MAX_FILE_BYTES + 1)
    if len(text.encode("utf-8")) > MAX_FILE_BYTES:
        raise ValueError("OBJ exceeds the 2 MB educational loader limit.")
    vertices, faces = [], []
    ignored = set()
    for line_number, line in enumerate(text.splitlines(), 1):
        fields = line.split("#", 1)[0].split()
        if not fields:
            continue
        try:
```

The excerpt comes from the current loader; later lines check face references and edge limits.

Appendix B | Code listings, continued

Listing 6. Opening an OBJ updates the model and recent list

tests/test_file_open.py, lines 20-31

```
    def test_open_obj_adds_selects_and_remembers_mesh(self):
        scene, models = Scene(), make_library()
        with tempfile.TemporaryDirectory() as temporary:
            settings = Path(temporary) / "settings.json"
            with patch("renderx.theme._SETTING", settings), \
                    patch("renderx.theme._recent_files", []):
                result = load_obj_file(SAMPLE_OBJ, scene, models)
                self.assertEqual(result, "Opened house.obj")
                self.assertEqual(scene.shape, "house")
                self.assertIn("house", models)
                self.assertEqual(json.loads(settings.read_text())["recent_files"],
                                 [str(SAMPLE_OBJ.resolve())])
```

Listing 7. Current fit-aware icon placement

renderx/ui/components/button.py, lines 8-30

```
def icon_button_rect(rect, label, icon, *, align="center"):
    """Return the shared safe icon placement for a real button rectangle.

    Center-aligned icon buttons use a centered mark when the label cannot fit.
    Wider buttons center the icon and label as one group; left-aligned buttons
    retain a consistent leading inset.
    """
    rect = pygame.Rect(rect)
    size = min(theme.METRICS["icon_medium"] if icon.startswith("shape:")
               else theme.METRICS["icon_small"], max(1, rect.height - 8))
    size = max(1, size)
    padding = theme.SPACE["md"]
    gap = theme.SPACE["sm"]
    label_width = fonts.font("control").size(str(label))[0]
    group_width = size + gap + label_width
    can_fit_group = group_width <= rect.width - padding * 2
    if align == "center" and can_fit_group:
        left = rect.centerx - group_width // 2
    elif align == "center" or not can_fit_group:
        left = rect.centerx - size // 2
    else:
        left = rect.x + padding
    return pygame.Rect(left, rect.centery - size // 2, size, size)
```

Listing 8. Earlier fixed leading placement from local history

renderx/ui/components/button.py at commit 98354b3, lines 29-37

```
    content = rect.inflate(-theme.SPACE["md"] * 2, 0)
    if icon:
        size = min(theme.METRICS["icon_small"], content.height)
        icon_rect = pygame.Rect(content.x, content.centery - size // 2, size, size)
        icons.draw_icon(surface, icon, icon_rect, label_color)
        content.x += size + theme.SPACE["sm"]
    if shortcut:
        hint_width = fonts.font("micro").size(shortcut)[0]
        fonts.draw_text(surface, shortcut, (rect.right - theme.SPACE["sm"], rect.centery),
```

The earlier snippet has no centering decision. The current helper and pixel-bound test are shown above.

Appendix C | Screenshot evidence

Current app captures below were made on 23 October 2026; the historical reference is labelled. The current Shooter capture shows the destination shell only, not live OpenGL gameplay.

Explore and file workflows

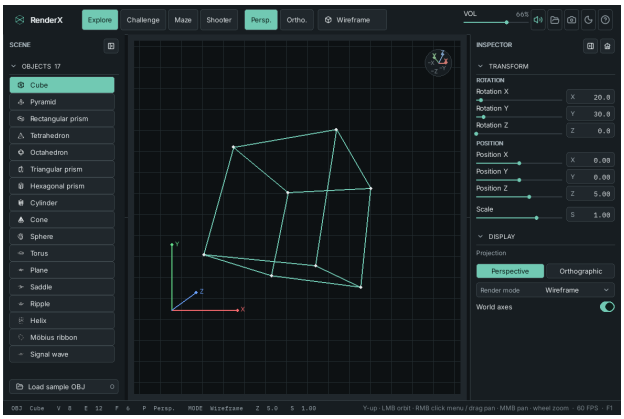


Figure 9. Explore with the built-in cube, scene list, inspector and status strip.

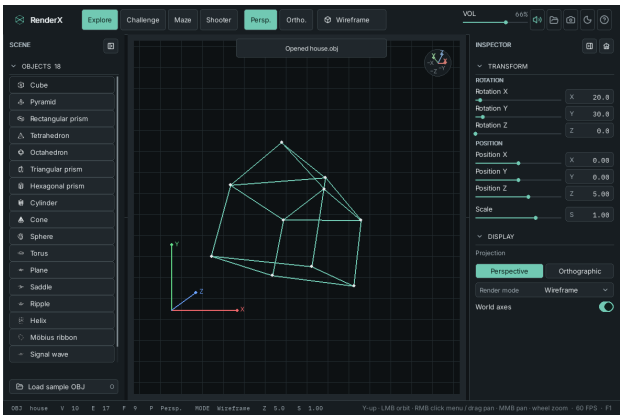


Figure 10. Bundled house OBJ loaded in the renderer.

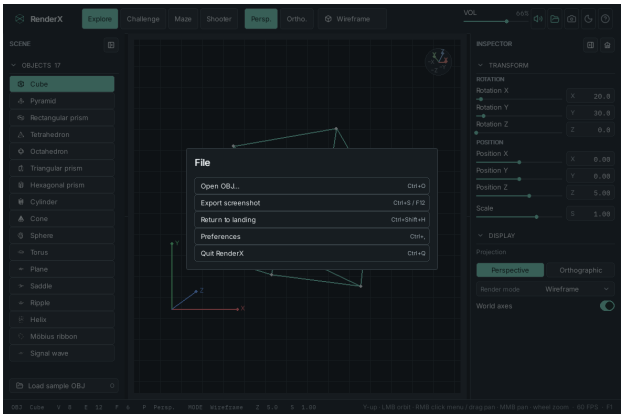


Figure 11. File menu and the available open actions.

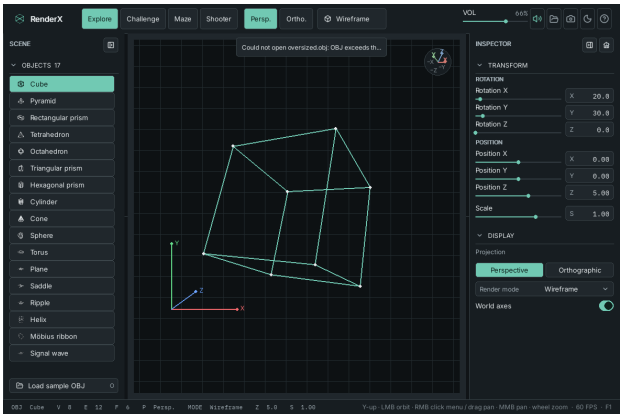


Figure 12. Readable feedback for an oversized OBJ.



Figure 13. Keyboard shortcut overlay.

Destinations and workspace controls

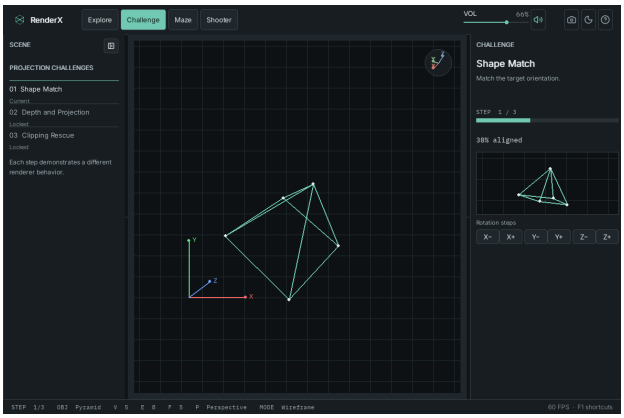


Figure 14. Challenges destination.

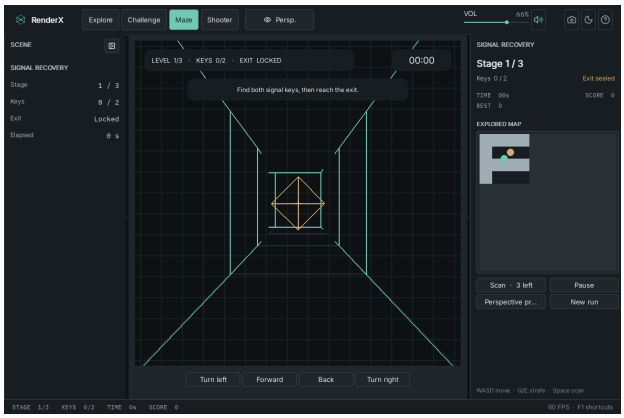


Figure 15. First-person wireframe Maze destination.

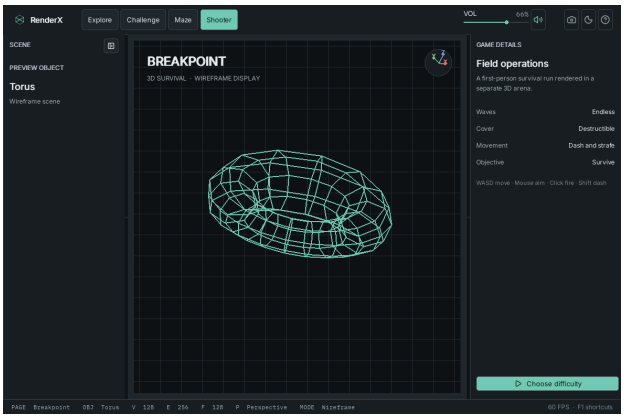


Figure 16. Shooter destination shell; not live OpenGL gameplay.

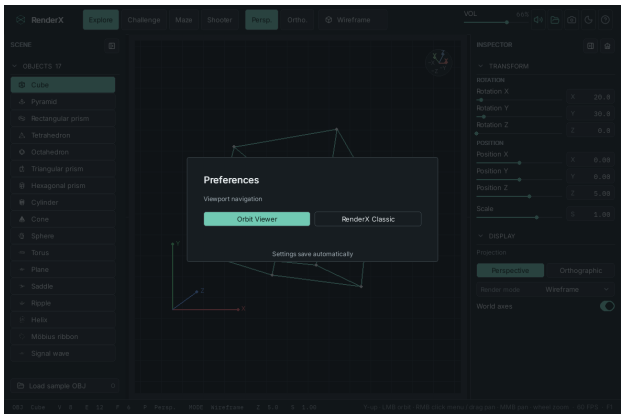


Figure 17. Preferences and audio controls.

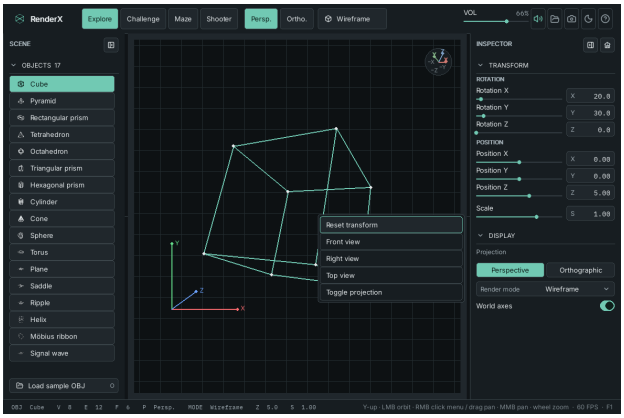


Figure 18. Viewport context menu in Explore.

Display sizes and an earlier project capture

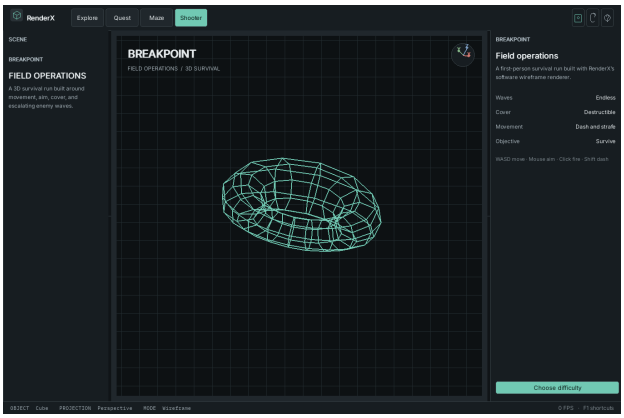


Figure 19. Earlier Breakpoint destination capture; historical reference, not from this validation run.

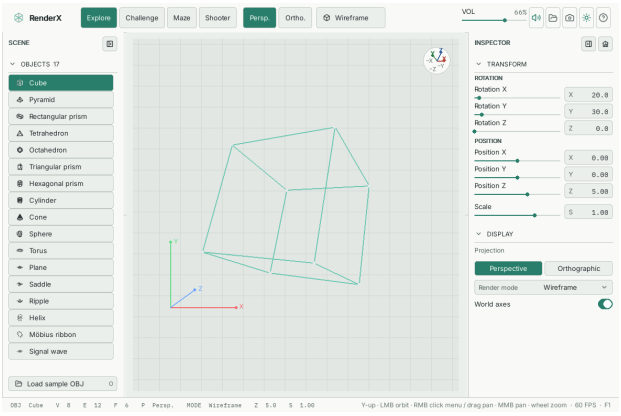


Figure 20. Explore in the alternate light theme.

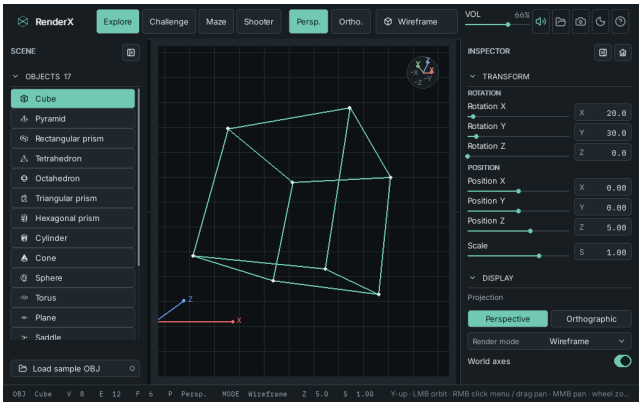


Figure 21. Compact Explore layout at 1024 by 640.

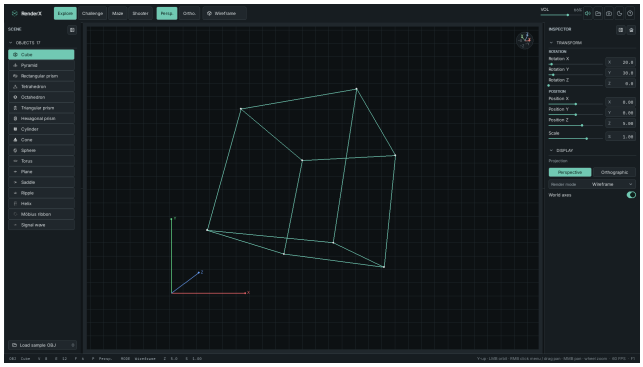


Figure 22. Explore layout at 1920 by 1080.

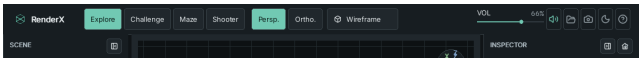


Figure 23. Enlarged crop from the real Explore toolbar capture.